PUC Minas

Testes de Software

Análise de Eficácia de Testes com Teste de Mutação

Aluno: Gabriel Reis Lebron de Oliveira

Matrícula: 855324

Repositório: https://github.com/GabrielReiiss/operacoes-mutante

Belo Horizonte, outubro de 2026

1. Análise Inicial

O projeto contém 50 funções matemáticas em `src/operacoes.js` e uma suíte de 50 testes Jest em `test/operacoes.test.js`. A cobertura foi medida com `npm test -- --coverage` e a pontuação de mutação com `npx stryker run`.

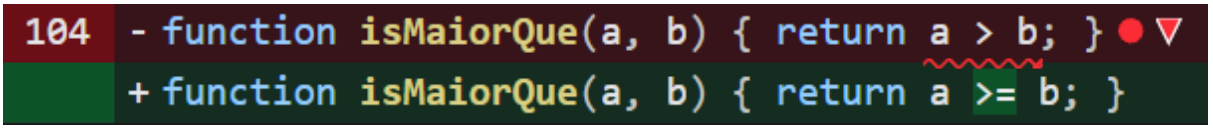
Métrica (suíte original)	Valor
Cobertura de linhas	98,64%
Cobertura de funções	100%
Cobertura de statements	85,41%
Cobertura de branches	58,82%
Pontuação de mutação	**73,71%**
Mutantes gerados	213
Mortos (Killed) + Timeout	154 + 3
Sobreviventes (Survived)	44
Sem cobertura (NoCoverage)	12

Discussão da discrepância. Quase todas as linhas (98,64%) e todas as funções são executadas pelos testes, mas 56 dos 213 bugs artificiais inseridos pelo Stryker passaram sem que nenhum teste falhasse. A cobertura apenas registra que uma linha foi executada; ela não verifica se a asserção é capaz de distinguir o comportamento correto de um incorreto. A suíte original testa cada função com um único valor "do caminho feliz": não testa valores de fronteira (a === b), casos que deveriam retornar false, entradas inválidas (zero, negativos, arrays vazios) nem dados em formatos menos convenientes (arrays desordenados ou de tamanho par). O resultado é uma suíte que parece completa pela cobertura, mas que deixaria passar diversos bugs reais.

2. Análise de Mutantes Críticos

Foram escolhidos três mutantes sobreviventes da primeira execução, cada um representando um tipo diferente de fraqueza da suíte.

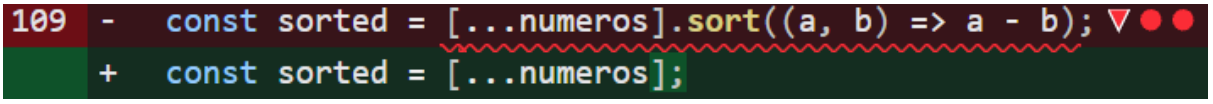
2.1 Operador relacional em isMaiorQue (linha 104)



O que a mutação fez: trocou o operador `>` por `>=`, fazendo a função considerar dois números iguais como "maior".

Por que sobreviveu: o único teste (nº 44) é `isMaiorQue(10, 5)`. Para essa entrada, `10 > 5` e `10 >= 5` são ambos true. Como o caso de fronteira `a === b` nunca é testado, a suíte não percebe a diferença. É o erro clássico de "off-by-one" em comparações.

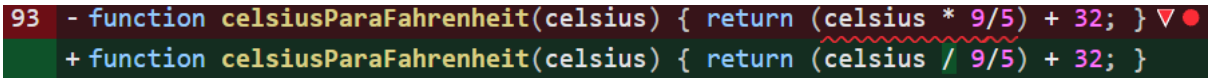
2.2 Remoção da ordenação em medianaArray (linha 109)



O que a mutação fez: removeu completamente a ordenação do array antes de calcular a mediana.

Por que sobreviveu: o teste nº 47 usa `medianaArray([1, 2, 3, 4, 5])`, um array que já está ordenado. Ordenar ou não ordenar produz o mesmo resultado (3). Este é o exemplo mais claro da diferença entre cobertura e eficácia: a linha está 100% coberta, mas o teste não verifica a responsabilidade principal dela. Também sobreviveram os mutantes que trocam o comparador por `a + b` ou por `() => undefined`, e o ramo para arrays de tamanho par nunca foi executado.

2.3 Operador aritmético em celsiusParaFahrenheit (linha 93)



O que a mutação fez: trocou a multiplicação por divisão, tornando a conversão completamente errada.

Por que sobreviveu: o teste nº 38 usa `celsiusParaFahrenheit(0)`. Com zero, qualquer multiplicação ou divisão resulta em zero, e as duas versões retornam 32. A entrada escolhida é "neutra" e anula a operação que deveria ser verificada. O mesmo ocorre em `fahrenheitParaCelsius(32)`, em que `32 - 32 = 0`.

3. Solução Implementada

Foram adicionados 24 testes (nº 51 a 74), cada um com uma única asserção, mantendo o estilo da suíte original. Os testes que matam os mutantes analisados são:

Mutante	Novo teste	Por que é eficaz
isMaiorQue: > → >=	`isMaiorQue(5, 5)` deve ser false (nº 69)	Testa exatamente a fronteira a === b, onde o original retorna false e o mutante retorna true.
medianaArray: sort removido	`medianaArray([5, 1, 3])` deve ser 3 (nº 73)	Sem ordenar, o elemento central seria 1. O array desordenado obriga a ordenação a funcionar.
medianaArray: ramo par	`medianaArray([4, 1, 3, 2])` deve ser 2.5 (nº 74)	Executa o ramo de tamanho par e verifica a média dos dois elementos centrais.
celsiusParaFahrenheit: * → /	`celsiusParaFahrenheit(100)` deve ser 212 (nº 66)	Com valor diferente de zero, o mutante retorna ≈ 34,22 em vez de 212.

Os demais testes seguem os mesmos princípios para os outros sobreviventes:

- **Fronteiras:** `raizQuadrada(0)`, `fatorial(0)`, `isPrimo(1)`, `isMenorQue(5, 5)`.

- **Casos que retornam false:** `isPar(7)`, `isImpar(4)`, `isPrimo(4)`, `isDivisivel(10, 3)`, `isEqual(7, 8)`.
- **Erros com verificação da mensagem:** entradas negativas, zero e arrays vazios usando `toThrow('trecho da mensagem')`, o que também mata os mutantes que apagam o texto do erro.
- **Valores fora do intervalo e arrays vazios:** `clamp(-5, 0, 10)`, `clamp(15, 0, 10)`, `mediaArray([])`.

Esses testes são eficazes porque cada um foi construído a partir de uma mutação específica: escolhe-se uma entrada em que o código original e o mutante **produzem resultados diferentes**, e a asserção verifica o valor exato. Todos os 74 testes passam com o código original.

4. Resultados Finais

Métrica	Antes	Depois
Número de testes	50	74
Pontuação de mutação	**73,71%**	**100%**
Mortos (Killed) + Timeout	154 + 3	199 + 3
Sobreviventes	44	0
Sem cobertura	12	0
Ignorados (Equivalentes)	0	11
Cobertura de branches	58,82%	97,05%
Cobertura de linhas	98,64%	100%

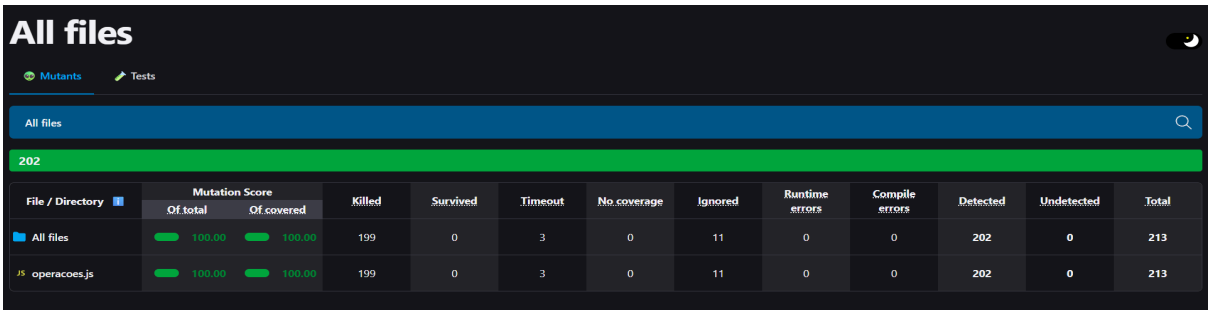
A pontuação de mutação subiu de **73,71% para 100%.** Somente com os 24 novos testes, a pontuação chegou a 96,71%: todos os mutantes foram mortos, exceto 7 mutantes equivalentes. A cobertura de linhas quase não mudou (de 98,64% para 100%), o que mostra que a melhoria foi na qualidade das asserções e não na quantidade de código executado.

Mutantes equivalentes. Os 7 mutantes que sobreviveram à execução sem os comentários são equivalentes: o código mutado se comporta igual ao original para qualquer entrada, então nenhum teste consegue matá-los:

- `fatorial` (linha 19, 4 mutantes): sem o `if (n === 0 || n === 1)`, o laço não executa e a função já retorna 1.
- `produtoArray` (linha 84): `[].reduce(..., 1)` já retorna 1 para array vazio.
- `clamp` (linhas 88 e 89): trocar `<` por `<=` (e `>` por `>=`) só muda o caso valor `===` limite, em que retornar o limite ou o valor dá o mesmo número.

Como nenhum teste é capaz de matar um mutante equivalente, eles foram marcados no código com o comentário “// Stryker disable next-line <mutador>: <justificativa>”, que os retira do cálculo como "Ignored" sem alterar o comportamento do código. Como o comentário desativa o tipo de mutação na linha inteira, foram ignorados 11 mutantes: os 7

equivalentes e outros 4 das mesmas linhas, que já eram mortos pelos testes. Com isso, a pontuação final é de 100%, acima da meta de 98%.



5. Conclusão

Este trabalho mostrou na prática que cobertura de código e eficácia de testes medem coisas diferentes. Com quase 99% de cobertura de linhas, a suíte original deixava passar um quarto dos bugs inseridos, incluindo erros graves, como remover a ordenação da mediana ou inverter uma conversão de temperatura. A cobertura responde "o código foi executado?", enquanto o teste de mutação responde "os testes perceberiam se o código estivesse errado?".

O teste de mutação também orienta a melhoria: cada mutante sobrevivente aponta uma entrada e uma asserção específicas que faltam, o que torna a escrita de testes objetiva. Por fim, os mutantes equivalentes mostram que a técnica tem limites e exige análise crítica: nem todo sobrevivente indica uma falha na suíte, e alguns revelam apenas código redundante. Mesmo com esse custo de análise e o tempo maior de execução, o teste de mutação é uma das formas mais confiáveis de avaliar a qualidade real de uma suíte de testes.